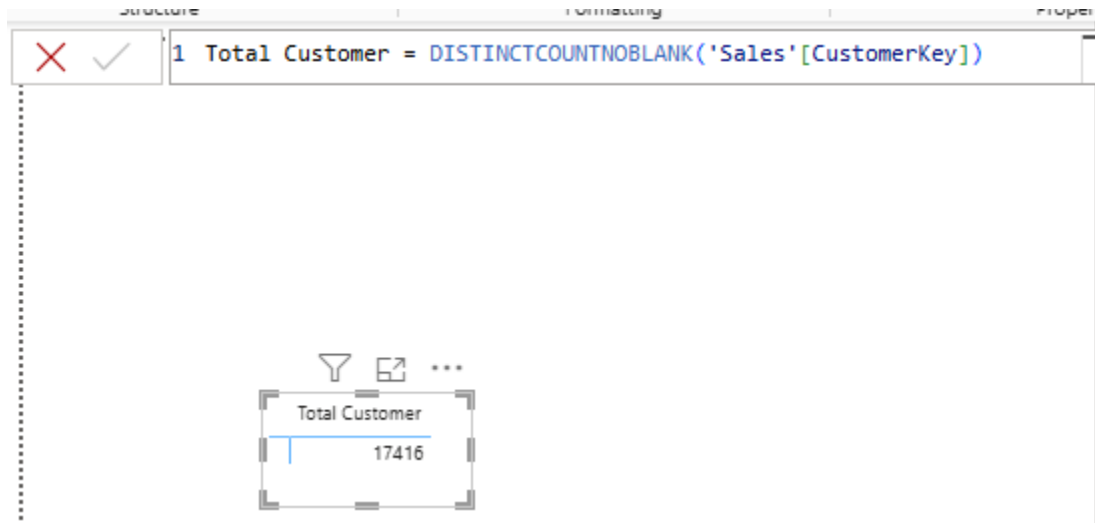


TASK-1:

Create a measure named Total Customers, to calculate the number of distinct AdventureWorks customers who made a transaction.

Output:



Create a measure named Return Rate, defined as quantity returned divided by quantity sold.

Output:

Return Rate

AdventureWorks S...

Format: Percentage

Data category: Unc...

Structure

Formatting

Property

```
1 Return Rate = Divide (  
2     SUM('AdventureWorks Returns Data'[ReturnQuantity]),  
3     SUM(Sales[OrderQuantity])  
4 )  
5
```

Return Rate	ProductName
3.42%	All-Purpose Bike Stand
1.11%	AWC Logo Cap
1.47%	Bike Wash - Dissolver
2.20%	Classic Vest, L
3.85%	Classic Vest, M
5.10%	Classic Vest, S
1.36%	Fender Set - Mountain
2.14%	Half-Finger Gloves, L
1.74%	Half-Finger Gloves, M
1.69%	Half-Finger Gloves, S
2.65%	Hitch Rack - 4-Bike
3.75%	HL Mountain Tire
3.52%	HL Road Tire
3.68%	Hydration Pack, 70 oz
2.17%	

A) Logical Function

Create a calculated column in the Customer Lookup table named Customer Priority If the customer is a parent and has an annual income > \$100,000, Custom Priority = Priority Otherwise, Customer Priority = Standard

Output:

The screenshot displays the Power BI Desktop interface with the 'Table' ribbon selected. The 'Name' field is set to 'Customer Priority', the 'Format' is 'Text', and the 'Data type' is 'Text'. The 'Summarization' is set to 'Don't summarize' and the 'Data category' is 'Uncategorized'. The DAX formula for the calculated column is as follows:

```
1 Customer Priority = IF(  
2     'AdventureWorks Customer Lookup'[MaritalStatus]="M"  
3     &&  
4     'AdventureWorks Customer Lookup'[TotalChildren]>=1  
5     &&  
6     'AdventureWorks Customer Lookup'[AnnualIncome]>100000,  
7     "Priority" , "Standard" )
```

Below the formula, a preview of the 'Customer Lookup' table is shown, displaying the 'CustomerKey' and 'Customer Priority' columns. The output shows that customers with a marital status of 'M' and a total of children greater than or equal to 1, and an annual income greater than 100,000, are assigned a 'Priority' status. Otherwise, they are assigned a 'Standard' status.

CustomerKey	Customer Priority
11081	Priority
11177	Priority
11179	Priority
11180	Priority
11181	Priority
11239	Priority
11240	Priority
11242	Priority
11243	Priority
11245	Priority
11248	Priority
11249	Priority
11250	Priority
11258	Priority
11260	Priority
11261	Priority
11268	Priority

Create a calculated column in the Customer Lookup table named Customer income: ○ If annual income is $\geq$ \$150,000, Very High If annual income is $\geq$ \$100,000, High If annual income is $\geq$ \$50,000, Average Otherwise, Income Level = Low

Output:

The screenshot displays the Power BI Desktop interface with a calculated column named 'Customer Income' in the 'Customer Lookup' table. The DAX formula is as follows:

```
1 Customer Income =  
2  
3 IF(  
4     'AdventureWorks Customer Lookup'[AnnualIncome]>=150000 , "VeryHigh",  
5     IF(  
6         'AdventureWorks Customer Lookup'[AnnualIncome]>=100000 , "High",  
7         IF(  
8             'AdventureWorks Customer Lookup'[AnnualIncome]>=50000 , "Average",  
9             "Low"  
10        )  
11    )  
12 )
```

Below the formula editor, a preview table shows the results for 16 rows of data:

CustomerKey	Customer Income
	Low
11000	Average
11001	Average
11002	Average
11003	Average
11004	Average
11005	Average
11007	Average
11008	Average
11009	Average
11010	Average
11011	Average
11012	High
11013	High
11014	High
11015	Low
11016	Low

Use a SWITCH function* to create another column named Education Category:

If EducationLevel is High School or Partial High School, Education Category = High School

If EducationLevel is Bachelors or Partial College, Education Category = Undergrad

If EducationLevel is Graduate Degree, Education Category = Graduate

Output:

The screenshot shows the Power BI DAX editor interface. The top ribbon includes tabs for Structure, Formatting, Properties, and Sort. The 'Name' field is set to 'Education Category', the 'Data type' is 'Text', and the 'Format' is 'Text'. The 'Summarization' is set to 'Don't summarize' and the 'Data category' is 'Uncategorized'. The main editor area contains the following DAX formula:

```
1 Education Category =  
2 SWITCH(  
3     TRUE(),  
4  
5     'AdventureWorks Customer Lookup'[EducationLevel] = "High School"  
6     || 'AdventureWorks Customer Lookup'[EducationLevel] = "Partial High School",  
7     "High School",  
8  
9     'AdventureWorks Customer Lookup'[EducationLevel] = "Bachelors"  
10    || 'AdventureWorks Customer Lookup'[EducationLevel] = "Partial College",  
11    "Undergrad",  
12  
13    'AdventureWorks Customer Lookup'[EducationLevel] = "Graduate",  
14    "Graduate"  
15 )
```

Below the formula editor, a preview table is displayed with the following data:

Education Category	CustomerKey
Undergrad	11000
Undergrad	11001
Undergrad	11002
Undergrad	11003
Undergrad	11004
Undergrad	11005
Undergrad	11007
Undergrad	11008
Undergrad	11009
Undergrad	11010
Undergrad	11011
Undergrad	11012

B) Text Function

Update the Month Short column in the Calendar Lookup table to extract and capitalize the first 3 characters of the month name

Output:

✕ ✓

1 Month Short =

2 UPPER(LEFT('Calendar'[Month Name], 3))

Date	Month Name	Month Number	Start of Year	Year	Month Short
01-01-2020 00:00:00	January	1	01-01-2020 00:00:00	2020	JAN
02-01-2020 00:00:00	January	1	01-01-2020 00:00:00	2020	JAN
03-01-2020 00:00:00	January	1	01-01-2020 00:00:00	2020	JAN
04-01-2020 00:00:00	January	1	01-01-2020 00:00:00	2020	JAN
05-01-2020 00:00:00	January	1	01-01-2020 00:00:00	2020	JAN
06-01-2020 00:00:00	January	1	01-01-2020 00:00:00	2020	JAN
07-01-2020 00:00:00	January	1	01-01-2020 00:00:00	2020	JAN
08-01-2020 00:00:00	January	1	01-01-2020 00:00:00	2020	JAN
09-01-2020 00:00:00	January	1	01-01-2020 00:00:00	2020	JAN
10-01-2020 00:00:00	January	1	01-01-2020 00:00:00	2020	JAN

Create a new column in the Product Lookup table named SKU Category, to return any number of characters before the first hyphen in the Product SKU column

Output:

✕ ✓ <pre>1 SKU Category = 2 LEFT(3 [ProductSKU], 4 FIND("-", [ProductSKU]) - 1 5)</pre> ^						
ProductColor	ProductSize	ProductStyle	ProductCost	ProductPrice	SKU Category	
Red	0	0	13.0863	34.99	HL	
Black	0	0	12.0278	33.6442	HL	
Blue	0	0	12.0278	33.6442	HL	
Multi	0	U	5.7052	8.6442	CA	
NA	0	0	65.8097	148.22	FK	
NA	0	0	77.9176	175.49	FK	
NA	0	0	101.8936	229.49	FK	
NA	0	0	15.1848	34.2	HS	

C)Date function

Create a new column in the Customer Lookup table named Birth Year,to extract only the year from the BirthDate column.

Output:

Structure		Formatting		Properties		Sort	Groups
				<pre>1 Birth Year = 2 YEAR('AdventureWorks Customer Lookup'[BirthDate])</pre>			
							
							
Qual	Y	Standard	Low	Undergrad	1965		
Qual	Y	Standard	Low	Undergrad	1963		
Qual	Y	Standard	Low	Undergrad	1975		
Qual	Y	Standard	Low	Undergrad	1972		
Qual	Y	Standard	Low	Undergrad	1972		
Qual	Y	Standard	Low	Undergrad	1969		
Qual	Y	Standard	Low	Undergrad	1974		
Qual	Y	Standard	Low	Undergrad	1963		
Qual	Y	Standard	Low	Undergrad	1963		
Qual	Y	Standard	Low	Undergrad	1963		

D) Calculate Function

Create a new measure named Bike Returns to calculate the total quantity of bikes returned

Output:

✖

✓

```
1 Bike Returns =  
2 CALCULATE(  
3     SUM('AdventureWorks Returns Data'[ReturnQuantity]),  
4     'AdventureWorks Product Subcategories Lookup'[ProductSubcategoryKey] IN {1, 2, 3}  
5 )
```

⌵

📄

...

Bike Returns	ProductSubcategoryKey
136	1
223	2
70	3
429	

Create a matrix to show Bike Returns (values) by Start of Month (rows). What do you notice about the volume of bike returns over time?

Output:

Month	Bike Returns
January	26
February	34
March	44
April	57
May	57
June	46
July	15
August	20
September	24
October	37
November	30
December	39
Total	429

Create a new measure named Bike Sales to calculate the total quantity of bikes sold, and add it to the matrix. What do you notice?

Output:

✖

✓

1

```
Bike Sales = CALCULATE( SUM(Sales[OrderQuantity]) ,  
'AdventureWorks Product Categories Lookup'[ProductCategoryKey]  
=1)
```

Month	Bike Sales
January	1192
February	1238
March	1352
April	1450
May	1651
June	1681
July	753
August	763
September	771
October	835
November	879
December	1364
Total	13929

Create a new measure named Bike Return Rate using either CALCULATE or DIVIDE, and add it to the matrix

Output:

✖

✓

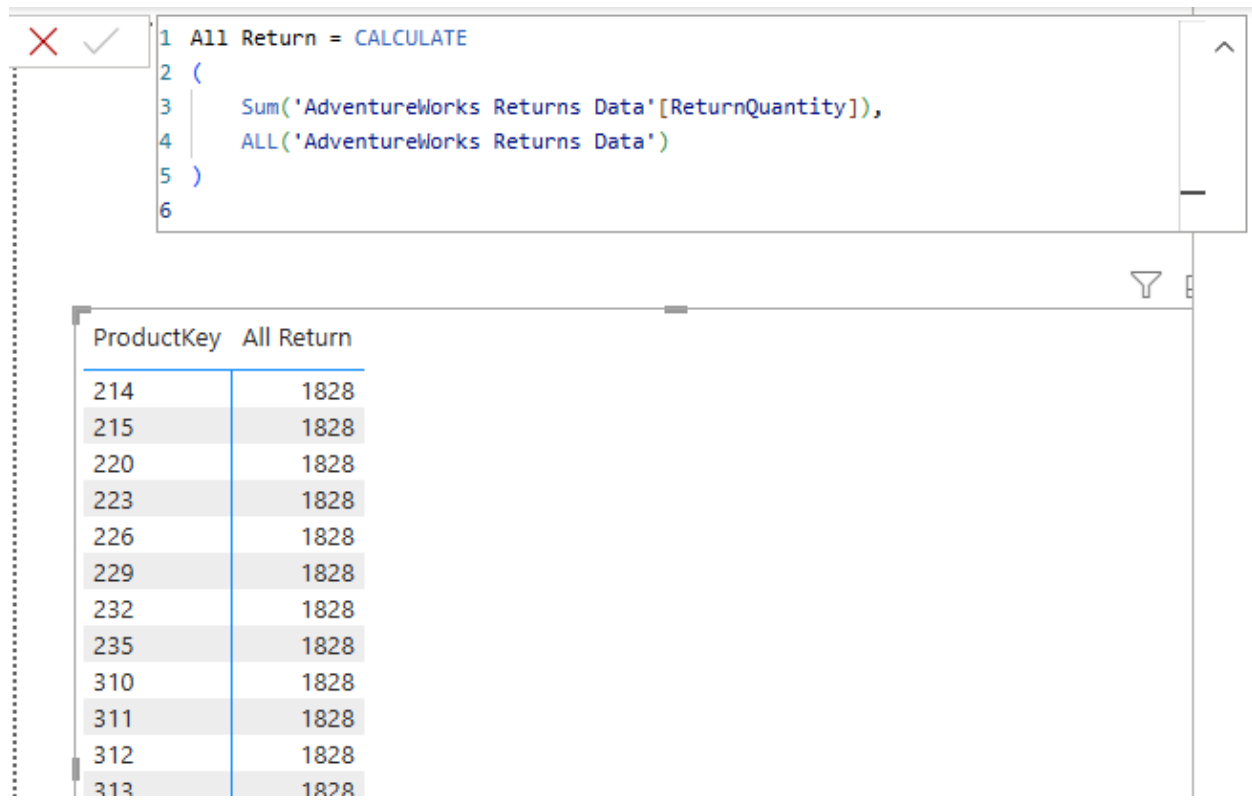
1 Bike Return Rate = DIVIDE([Bike Returns] , [Bike Sales])

Month	Bike Return Rate
January	0.02
February	0.03
March	0.03
April	0.04
May	0.03
June	0.03
July	0.02
August	0.03
September	0.03
October	0.04
November	0.03
December	0.03
Total	0.03

E) Calculate & All function

Create a new measure named All Returns to calculate the total number of returns, regardless of filter context

Output:



The screenshot shows a DAX formula bar with the following code:

```
1 All Return = CALCULATE
2 (
3     Sum('AdventureWorks Returns Data'[ReturnQuantity]),
4     ALL('AdventureWorks Returns Data')
5 )
6
```

Below the formula bar, a table displays the results of the measure. The table has two columns: 'ProductKey' and 'All Return'. The 'All Return' column shows a constant value of 1828 for all product keys, indicating that the measure successfully ignores the filter context.

ProductKey	All Return
214	1828
215	1828
220	1828
223	1828
226	1828
229	1828
232	1828
235	1828
310	1828
311	1828
312	1828
313	1828

Create a new measure named % of All Returns that divides Total Returns by All Returns

Output:

✖

✓

1 All Return Rate = `DIVIDE( [Total Return], [All Return])`

All Return	All Return Rate	Total Return
1828	0.62	1130

CategoryName

Access...

Bikes

Clothing

Compo...

Create a matrix to show % of All Returns (values) by product Category Name (rows).

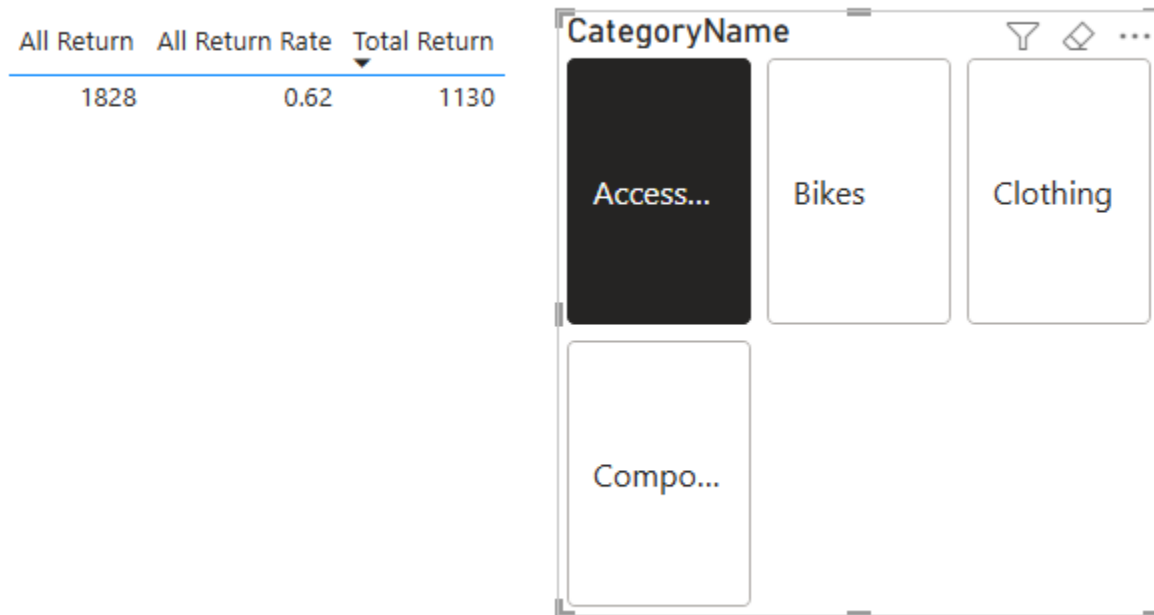
Output:

All Return	All Return Rate	Total Return	Category Name		
1828	0.15	269	Access...	Bikes	Clothing
			Compo...		

Which category accounts for the largest percentage of returns?

Accessories

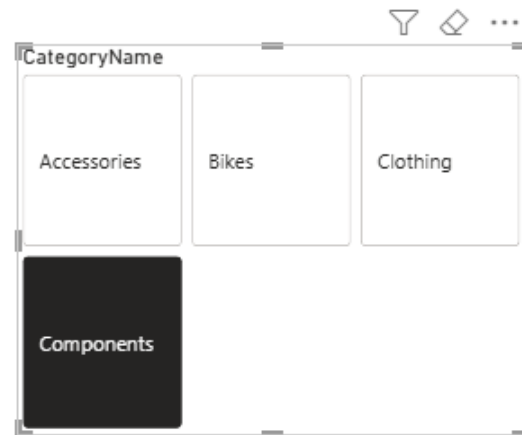
Output:



The smallest: It is of component category: 0 percent return

Output:

Total Return	All Return	All Return Rate
1828		



F) Iterator Function

Create a new measure named Total Cost that multiplies the order quantities in the Sales Data table by the product cost in the Product Lookup table, then calculates the sum

Output:

✖ ✓	
1 Total Sales =	
2 SUMX(	
3 Sales,	
4 Sales[OrderQuantity] * RELATED('AdventureWorks	
Product Lookup'[ProductPrice]	
5)	
Total Sales	Product Name
\$73,444.0	Sport-100 Helmet, Red
\$49,949.8	Touring Tire
\$13,672.6	Touring Tire Tube
\$4,19,596.3	Touring-1000 Blue, 46
\$3,52,842.4	Touring-1000 Blue, 50
\$3,74,299.0	Touring-1000 Blue, 54
\$3,50,458.3	Touring-1000 Blue, 60
\$4,00,523.8	Touring-1000 Yellow, 46
\$3,48,074.2	Touring-1000 Yellow, 50
\$3,64,762.7	Touring-1000 Yellow, 54
\$3,29,001.7	Touring-1000 Yellow, 60
\$1,16,625.6	Touring-2000 Blue, 46
\$1,21,485.0	Touring-2000 Blue, 50
\$1,04,477.1	Touring-2000 Blue, 54
\$95,973.2	Touring-2000 Blue, 60
\$38,602.2	Touring-3000 Blue, 44
\$35,632.8	Touring-3000 Blue, 50
\$40,086.9	Touring-3000 Blue, 54
\$42,313.9	Touring-3000 Blue, 58
\$46,768.0	Touring-3000 Blue, 62
\$43,798.6	Touring-3000 Yellow, 44
\$41,571.6	Touring-3000 Yellow, 50
\$34,890.4	Touring-3000 Yellow, 54
\$34,148.1	Touring-3000 Yellow, 58
\$35,632.8	Touring-3000 Yellow, 62
\$39,755.3	Water Bottle - 30 oz.
\$23,376.7	Women's Mountain Shorts, L
\$22,886.7	Women's Mountain Shorts, M
\$19,807.2	Women's Mountain Shorts, S
\$2,49,14,586.8	

Create a new measure named Total Profit (revenue minus cost)

Output:

Name

Total Profit

Format

Currency

Data category

Home table

Sales

\$

%

1

Structure

Formatting

1

Total Profit = [Total Sales] - [Total Cost]

</

Create a matrix to show Total Profit (values) by Year (rows).

Output:

Year	Total Profit
2020	\$26,01,602.3
2021	\$39,67,084.1
2022	\$38,89,029.0
Total	\$1,04,57,715.4

How much profit has Adventure Works earned so far in 2022

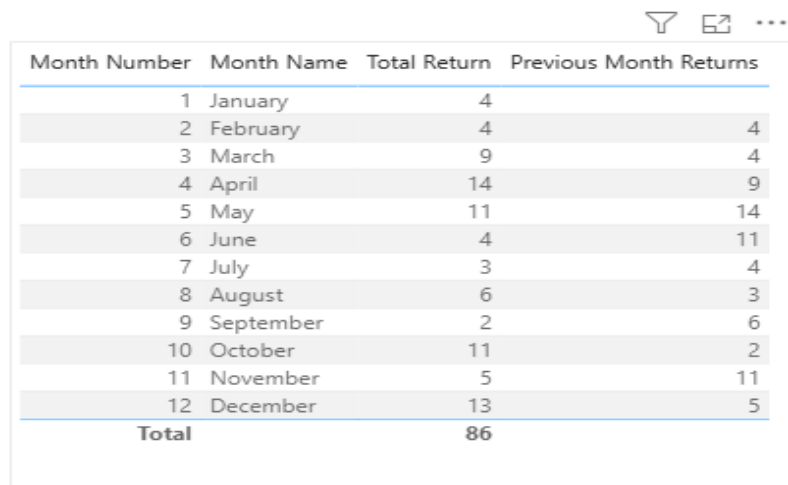
Output:

Total Profit	Year		
\$38,89,029.0	2020	2021	2022

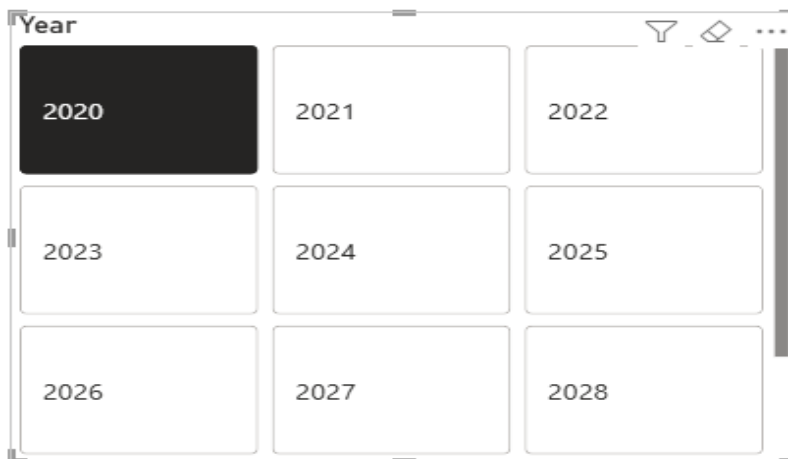
G) Time Intelligence Function

Previous Month Returns

Output:



Month Number	Month Name	Total Return	Previous Month Returns
1	January	4	
2	February	4	4
3	March	9	4
4	April	14	9
5	May	11	14
6	June	4	11
7	July	3	4
8	August	6	3
9	September	2	6
10	October	11	2
11	November	5	11
12	December	13	5
Total		86	



Year		
2020	2021	2022
2023	2024	2025
2026	2027	2028

Previous Month Orders

Output:

```
1 Previous Month Orders =
2 CALCULATE(
3     [Total Orders],
4     PREVIOUSMONTH('Calendar'[Date])
5 )
```

Year

2020

2021

2022

2023

2024

2025

2026

2027

2028

Month Number	Month Name	Total Orders	Previous Month Orders
1	January	184	
2	February	165	184
3	March	198	165
4	April	204	198
5	May	206	204
6	June	212	206
7	July	247	212
8	August	278	247
9	September	196	278
10	October	223	196
11	November	191	223
12	December	326	191
Total		2630	

Previous Month Profit

Output:

```
1 Previous Month Profit =
2 CALCULATE(
3     [Total Profit],
4     PREVIOUSMONTH('Calendar'[Date])
5 )
```

Year

2020

2021

2022

2023

2024

2025

2026

2027

2028

Month Number	Month Name	Previous Month Profit	Total Profit
1	January		\$2,35,814.0
2	February	\$2,35,814.0	\$2,12,186.7
3	March	\$2,12,186.7	\$2,59,084.5
4	April	\$2,59,084.5	\$2,63,031.3
5	May	\$2,63,031.3	\$2,66,275.8
6	June	\$2,66,275.8	\$2,70,067.5
7	July	\$2,70,067.5	\$1,96,682.8
8	August	\$1,96,682.8	\$2,18,355.5
9	September	\$2,18,355.5	\$1,40,516.1
10	October	\$1,40,516.1	\$1,68,581.8
11	November	\$1,68,581.8	\$1,34,176.0
12	December	\$1,34,176.0	\$2,36,830.3
Total			\$26,01,602.3

Order Target (10% increase over previous month)

Output:

```
1 Order Target =  
2 IF(  
3   [Total Orders] >= [Previous Month Orders] * 1.10,  
4   "Yes",  
5   "No"  
6 )
```

Year

2020	2021	2022
2023	2024	2025
2026	2027	2028

Month Number	Month Name	Total Orders	Order Target	Previous Month Orders
1	January	184	Yes	
2	February	165	No	184
3	March	198	Yes	165
4	April	204	No	198
5	May	206	No	204
6	June	212	No	206
7	July	247	Yes	212
8	August	278	Yes	247
9	September	196	No	278
10	October	223	Yes	196
11	November	191	No	223
12	December	326	Yes	191
Total		2630	Yes	

Profit Target (10% increase over previous month)

Output:

```
1 Previous Month Profit =
2 CALCULATE(
3     [Total Profit],
4     PREVIOUSMONTH('Calendar'[Date])
5 )
```

Year	Month	Previous Month Profit	Total Profit	Profit Target
2020	January		\$2,35,814.0	Yes
2020	February	\$2,35,814.0	\$2,12,186.7	No
2020	March	\$2,12,186.7	\$2,59,084.5	Yes
2020	April	\$2,59,084.5	\$2,63,031.3	No
2020	May	\$2,63,031.3	\$2,66,275.8	No
2020	June	\$2,66,275.8	\$2,70,067.5	No
2020	July	\$2,70,067.5	\$1,96,682.8	No
2020	August	\$1,96,682.8	\$2,18,355.5	Yes
2020	September	\$2,18,355.5	\$1,40,516.1	No
2020	October	\$1,40,516.1	\$1,68,581.8	Yes
2020	November	\$1,68,581.8	\$1,34,176.0	No
2020	December	\$1,34,176.0	\$2,36,830.3	Yes
2021	January	\$2,36,830.3	\$1,82,044.4	No
2021	February	\$1,82,044.4	\$2,00,044.4	No
2021	March	\$2,00,044.4	\$1,99,611.0	No
2021	April	\$1,99,611.0	\$2,09,521.7	No
2021	May	\$2,09,521.7	\$2,33,013.1	Yes
2021	June	\$2,33,013.1	\$2,27,745.0	No
2021	July	\$2,27,745.0	\$3,42,624.1	Yes
2021	August	\$3,42,624.1	\$3,48,095.7	No
2021	September	\$3,48,095.7	\$4,10,592.1	Yes
2021	October	\$4,10,592.1	\$4,41,168.0	No
2021	November	\$4,41,168.0	\$4,82,940.4	No
2021	December	\$4,82,940.4	\$6,89,684.2	Yes
2022	January	\$6,89,684.2	\$5,41,843.8	No
Total			\$1,04,57,715.4	Yes

90-day Rolling Profit:

Output:

```
1 90 Day Rolling Profit =  
2 CALCULATE(  
3     [Total Profit],  
4     DATESINPERIOD(  
5         'Calendar'[Date],  
6         MAX('Calendar'[Date]),  
7         -90,  
8         DAY  
9     )  
10 )
```

Year	Month	Day	Total Profit	90 Day Rolling Profit
2020	January	1	\$3,455.8	3,455.78
2020	January	2	\$5,627.9	9,083.68
2020	January	3	\$11,476.5	20,560.18
2020	January	4	\$7,115.7	27,675.92
2020	January	5	\$3,099.9	30,775.83
2020	January	6	\$8,511.8	39,287.60
2020	January	7	\$3,385.9	42,673.46
2020	January	8	\$10,285.6	52,959.02
2020	January	9	\$5,627.9	58,586.92
2020	January	10	\$5,697.8	64,284.74
2020	January	11	\$12,883.5	77,168.23
2020	January	12	\$9,848.8	87,017.06
2020	January	13	\$3,099.9	90,116.96
2020	January	14	\$12,905.4	1,03,022.32
2020	January	15	\$9,018.4	1,12,040.75
2020	January	16	\$9,999.6	1,22,040.36
2020	January	17	\$10,285.6	1,32,325.92
2020	January	18	\$6,199.8	1,38,525.73
2020	January	19	\$7,320.8	1,45,846.56
2020	January	20	\$8,441.9	1,54,288.41
Total			\$1,04,57,715.4	